

JAVA

RECURSION PROBLEMS

Detailed Notes • Java Code • Dry Runs • Complexity • Practice

Beginner → Strong Recursion Foundation

The 3 questions to ask before writing recursion

1. Base case: When should the function stop?
2. Smaller problem: What smaller version of the same problem can I solve?
3. Progress: Does every call move toward the base case?

Call stack

Every active recursive call has its own stack frame containing parameters and local variables. Frames are pushed during calls and popped during returns. Too many calls can cause `StackOverflowError`.

Contents

No.	Topic
01	Understand recursion by printing something N times
02	Print name N times using recursion
03	Print 1 to N using recursion
04	Print N to 1 using recursion
05	Sum of First N Numbers
06	Factorial
07	Reverse an Array
08	String Palindrome
09	Fibonacci Number
10	Call Stack & Recursion Flow
11	Common Recursion Patterns
12	Recursion Practice Questions

01

Understand Recursion by Printing Something N Times

Print once, decrease n, and call the same function until n reaches zero.

KEY LOGIC / FORMULA

Base case: $n == 0$

Recursive step: print $\rightarrow$ solve($n-1$)

Java implementation

```
static void printSomething(int n) {  
    if (n == 0) return;  
    System.out.println("Hello");  
    printSomething(n - 1);  
}
```

Dry run

$n=3 \rightarrow$ Hello $\rightarrow n=2 \rightarrow$ Hello $\rightarrow n=1 \rightarrow$ Hello $\rightarrow n=0 \rightarrow$ stop.

COMPLEXITY

$O(n)$ time, $O(n)$ stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Print Name N Times Using Recursion

Same pattern, but the printed value is a name.

KEY LOGIC / FORMULA

if $n == 0$ return; $\rightarrow$ print $\rightarrow n-1$

Java implementation

```
static void printName(int n) {  
    if (n == 0) return;  
    System.out.println("Subha");  
    printName(n - 1);  
}
```

Dry run

$n=3$ prints the name 3 times. The important part is that n decreases on every call.

COMPLEXITY

$O(n)$ time, $O(n)$ stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

03

Print 1 to N Using Recursion

Recurse first and print during the return phase.

KEY LOGIC / FORMULA

solve(n-1) → print n

Java implementation

```
static void print1ToN(int n) {  
    if (n == 0) return;  
    print1ToN(n - 1);  
    System.out.println(n);  
}
```

Dry run

n=4 reaches 0 first, then prints 1, 2, 3, 4 while the stack unwinds.

COMPLEXITY

O(n) time, O(n) stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Print N to 1 Using Recursion

Print before the recursive call, so output happens while going down the stack.

KEY LOGIC / FORMULA

print n $\rightarrow$ solve(n-1)

Java implementation

```
static void printNTol(int n) {  
    if (n == 0) return;  
    System.out.println(n);  
    printNTol(n - 1);  
}
```

Dry run

n=4 $\rightarrow$ 4, 3, 2, 1.

COMPLEXITY

O(n) time, O(n) stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Sum of First N Numbers

Use the recurrence $\text{sum}(n) = n + \text{sum}(n-1)$, with $\text{sum}(0) = 0$.

KEY LOGIC / FORMULA

$\text{sum}(n) = n + \text{sum}(n-1)$
 $\text{sum}(0) = 0$

Java implementation

```
static int sum(int n) {  
    if (n == 0) return 0;  
    return n + sum(n - 1);  
}
```

Dry run

$\text{sum}(4) = 4 + 3 + 2 + 1 = 10$.

COMPLEXITY

$O(n)$ time, $O(n)$ stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Factorial of a Given Number

Factorial has a natural recursive definition: $n! = n \times (n-1)!$.

KEY LOGIC / FORMULA

```
fact(n)=n*fact(n-1)
fact(0)=1
```

Java implementation

```
static long factorial(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}
```

Dry run

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

COMPLEXITY

$O(n)$ time, $O(n)$ stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Reverse an Array

Use two pointers. Swap the outer elements, then move inward.

KEY LOGIC / FORMULA

base: $\text{left} \geq \text{right}$
recursive: $\text{left}+1, \text{right}-1$

Java implementation

```
static void reverse(int[] a, int left, int right) {  
    if (left >= right) return;  
    int temp=a[left]; a[left]=a[right]; a[right]=temp;  
    reverse(a, left+1, right-1);  
}
```

Dry run

$[1,2,3,4,5] \rightarrow \text{swap } 1/5 \rightarrow \text{swap } 2/4 \rightarrow [5,4,3,2,1]$.

COMPLEXITY

$O(n)$ time, $O(n)$ stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Check if String is Palindrome

Compare the first and last characters. If equal, move inward.

KEY LOGIC / FORMULA

base: left $\geq$ right $\rightarrow$ true
mismatch $\rightarrow$ false

Java implementation

```
static boolean palindrome(String s, int l, int r) {  
    if (l >= r) return true;  
    if (s.charAt(l) != s.charAt(r)) return false;  
    return palindrome(s, l+1, r-1);  
}
```

Dry run

level $\rightarrow$ l/l, e/e, v/v $\rightarrow$ true. hello $\rightarrow$ h/o mismatch $\rightarrow$ false.

COMPLEXITY

$O(n)$ time, $O(n)$ stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Fibonacci Number

Fibonacci is defined recursively as $F(n)=F(n-1)+F(n-2)$.

KEY LOGIC / FORMULA

$F(0)=0$, $F(1)=1$

$F(n)=F(n-1)+F(n-2)$

Java implementation

```
static int fib(int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return fib(n-1) + fib(n-2);  
}
```

Dry run

$\text{fib}(5)=5$. The naive version repeats the same subproblems many times.

COMPLEXITY

Time $\approx O(2^n)$, stack $O(n)$. Memoization can reduce time to $O(n)$.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Call Stack & Recursion Flow

Code before recursion runs while the stack grows. Code after recursion runs while the stack unwinds.

KEY LOGIC / FORMULA

CALL → CALL → BASE CASE → RETURN → RETURN

Java implementation

```
static void fun(int n) {  
    if (n == 0) return;  
    System.out.println("Before " + n);  
    fun(n-1);  
    System.out.println("After " + n);  
}
```

Dry run

n=3: Before 3, Before 2, Before 1, After 1, After 2, After 3.

COMPLEXITY

Depth $n \rightarrow O(n)$ stack space.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

Common Recursion Patterns

Most beginner problems use one of three patterns: decrease n , move two pointers, or branch into multiple calls.

KEY LOGIC / FORMULA

Pattern A: $n-1$

Pattern B: $\text{left}+1/\text{right}-1$

Pattern C: two recursive calls

Java implementation

```
// A
solve(n-1);

// B
solve(left+1, right-1);

// C
solve(n-1);
solve(n-2);
```

Dry run

Printing uses A, reverse/palindrome uses B, Fibonacci uses C.

COMPLEXITY

Analyze number of calls + work per call.

COMMON TIP

Always identify the base case first. Then verify that every recursive call makes the input smaller or otherwise moves toward termination.

100 Recursion Practice Questions

Try these in Java. The list moves from basic recursion mechanics to arrays, strings, Fibonacci, optimization, recursion trees, and interview concepts.

LEVEL 1 — BASIC

#	Question	Level
1	Print a message N times.	Easy
2	Print your name N times.	Easy
3	Print 1 to N.	Easy
4	Print N to 1.	Easy
5	Print N to 1 and then 1 to N.	Easy
6	Print 1 to N and then N to 1.	Easy
7	Print even numbers from 1 to N.	Easy
8	Print odd numbers from 1 to N.	Easy
9	Print multiples of 3 up to N.	Easy
10	Count down from N to 0.	Easy

LEVEL 2 — SUM & MATH

#	Question	Level
11	Sum first N natural numbers.	Easy
12	Sum first N even numbers.	Easy
13	Sum first N odd numbers.	Easy
14	Factorial of N.	Easy
15	Calculate x^n recursively.	Easy
16	Product of numbers 1 to N.	Easy
17	Sum of digits recursively.	Easy
18	Count digits recursively.	Easy
19	Product of digits recursively.	Easy
20	Digital root recursively.	Easy

LEVEL 3 — NUMBERS

#	Question	Level
21	Reverse a number recursively.	Easy-Medium
22	Check number palindrome.	Easy-Medium
23	Find first digit recursively.	Easy-Medium
24	Find largest digit recursively.	Easy-Medium
25	Find smallest digit recursively.	Easy-Medium
26	Count zeros in a number.	Easy-Medium
27	Count occurrences of a digit.	Easy-Medium
28	Sum even digits.	Easy-Medium
29	Sum odd digits.	Easy-Medium
30	Check Armstrong number recursively.	Easy-Medium

LEVEL 4 — ARRAYS

#	Question	Level
31	Print array recursively.	Easy-Medium
32	Print array in reverse.	Easy-Medium
33	Find array sum.	Easy-Medium
34	Find array maximum.	Easy-Medium
35	Find array minimum.	Easy-Medium
36	Find array product.	Easy-Medium
37	Count even elements.	Easy-Medium
38	Count odd elements.	Easy-Medium
39	Recursive linear search.	Easy-Medium
40	Find first index of target.	Easy-Medium

LEVEL 5 — ARRAY ADVANCED

#	Question	Level
41	Find last index of target.	Easy-Medium
42	Check if array is sorted.	Easy-Medium
43	Reverse array with two pointers.	Easy-Medium
44	Find second-largest element.	Easy-Medium
45	Count target occurrences.	Easy-Medium
46	Check if two arrays are equal.	Easy-Medium
47	Sum elements at even indices.	Easy-Medium
48	Sum elements at odd indices.	Easy-Medium
49	Find maximum adjacent difference.	Easy-Medium
50	Remove adjacent duplicates recursively.	Easy-Medium

LEVEL 6 — STRINGS

#	Question	Level
51	Print string characters recursively.	Medium
52	Reverse a string.	Medium
53	Check string palindrome.	Medium
54	Count vowels.	Medium
55	Count consonants.	Medium
56	Count spaces.	Medium
57	Remove all occurrences of a character.	Medium
58	Replace a character recursively.	Medium
59	Find first occurrence of a character.	Medium
60	Find last occurrence of a character.	Medium

LEVEL 7 — CLASSIC RECURSION

#	Question	Level
61	Nth Fibonacci.	Medium
62	Print first N Fibonacci numbers.	Medium
63	Sum first N Fibonacci numbers.	Medium
64	Nth Tribonacci.	Medium
65	Count ways to climb N stairs.	Medium
66	Ways using 1 or 2 steps.	Medium
67	Tower of Hanoi.	Medium
68	Generate binary strings of length N.	Medium
69	Generate all subsequences of a string.	Medium
70	Generate all subsequences of an array.	Medium

LEVEL 8 — RECURSION TREE

#	Question	Level
71	Draw recursion tree for fib(5).	Medium
72	Count calls made by fib(n).	Medium
73	Find maximum recursion depth of fib(n).	Medium
74	Trace sum(5).	Medium
75	Trace factorial(5).	Medium
76	Trace print1ToN(5) including unwind.	Medium
77	Predict output for code before/after recursion.	Medium
78	Explain exponential recursion.	Medium
79	Identify base cases in recursive code.	Medium
80	Identify tail vs non-tail recursion.	Medium

LEVEL 9 — OPTIMIZATION

#	Question	Level
81	Convert recursive factorial to iteration.	Hard
82	Convert recursive sum to iteration.	Hard
83	Convert Fibonacci to iteration.	Hard
84	Memoize Fibonacci.	Hard
85	Store Fibonacci results in an array.	Hard
86	Compare naive and memoized Fibonacci.	Hard
87	Calculate x^n in $O(\log n)$.	Hard
88	Recursive Euclidean GCD.	Hard
89	LCM using recursive GCD.	Hard
90	Convert decimal to binary recursively.	Hard

LEVEL 10 — INTERVIEW / CHALLENGE

#	Question	Level
91	Explain why recursion uses stack memory.	Hard
92	Explain StackOverflowError.	Hard
93	Explain why call frames are on the stack.	Hard
94	Complexity of factorial recursion.	Hard
95	Complexity of recursive array reversal.	Hard
96	Write merge-sort recurrence.	Hard
97	Explain recursive binary search $O(\log n)$.	Hard
98	Generate permutations of a string.	Hard
99	Generate combinations recursively.	Hard
100	Solve a simple maze using recursion/backtracking.	Hard

Pattern	Core idea	Complexity
N times	solve(n-1)	$O(n)$ / $O(n)$ stack
1 to N	solve first, print later	$O(n)$ / $O(n)$
N to 1	print first, solve later	$O(n)$ / $O(n)$
Sum	$n + \text{sum}(n-1)$	$O(n)$ / $O(n)$
Factorial	$n \times \text{fact}(n-1)$	$O(n)$ / $O(n)$
Array reverse	left++, right--	$O(n)$ / $O(n)$
Palindrome	compare ends, move inward	$O(n)$ / $O(n)$
Fibonacci	$\text{fib}(n-1) + \text{fib}(n-2)$	$\approx O(2^n)$ / $O(n)$
Binary search	half the range	$O(\log n)$ / $O(\log n)$
GCD	$\text{gcd}(b, a \% b)$	$O(\log \min)$

Remember

Base case → Smaller problem → Progress toward base case.

Practice tracing both phases: going down the recursion and coming back up the call stack.

End of Recursion Problems — Java Notes + 100 Practice Questions